



Django

Capítulo 8 Trabalhando com Templates II

Django

Capítulo 8 - Trabalhando com Templates II

O Django tem um modelo de usuário nativo, chamado de User que contém os campos básicos de definição de usuários, tais como nome do usuário (username), senha (password), e-mail (email), data de criação (data_joined). Entretanto, em geral, é necessário criar uma tabela adicional para incluir informações específicas vinculadas aos controles da aplicação. Podemos citar dentre informações específicas o perfil de acesso (usuário comum ou administrador), endereço e data de aniversário. Abaixo temos a estrutura da tabela de usuários nativa que é criada no banco e dados como auth_user.

	Nome	Tipo de dado	Primary Key	Foreign Key	Unique	Verificar	Not NULL	Collate	Gerado	Valor padrão
1	id	integer								NULL
2	password	varchar (128)								NULL
3	last_login	datetime								NULL
4	is_superuser	bool								NULL
5	username	varchar (150)								NULL
6	last_name	varchar (150)								NULL
7	email	varchar (254)								NULL
8	is_staff	bool								NULL
9	is_active	bool								NULL
10	date_joined	datetime								NULL
11	first_name	varchar (150)								NULL

Django

Capítulo 8 - Trabalhando com Templates II

No exemplo que estamos montando incluiremos a entidade Usuario para acrescentar as informações adicionais de usuário. Para isso Criaremos a classe modelo de usuário.

Na pasta loja\models crie o arquivo Usuario.py e adicione o código a seguir.

Parte 1 do conteúdo do loja/models/**Usuario.py**

```
from loja.models import *
class Usuario(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    perfil = models.IntegerField(choices=PERFIL, default=2)
    aniversario = models.DateField(default=None, null=True, blank=True)
    criado_em = models.DateTimeField(auto_now_add=True)
    alterado_em = models.DateTimeField(auto_now=True)
    token = models.CharField(max_length=255, null=True, blank=True)
    def __str__(self):
        return '{}'.format(self.user.username)
    @receiver(post_save, sender=User)
```

Django

Capítulo 8 - Trabalhando com Templates II

Parte 2 do conteúdo do loja/models/**Usuario.py**

```
def create_user_usuario(sender, instance, created, **kwargs):
    try:
        if created:
            Usuario.objects.create(user=instance)
    except:
        pass
@receiver(post_save, sender=User)
def save_user_usuario(sender, instance, **kwargs):
    try:
        instance.usuario.save()
    except:
        pass
```

Django

Capítulo 8 - Trabalhando com Templates II

Imagem do conteúdo do loja/models/**Usuario.py**

```
loja > models > Usuario.py > ...
1  from loja.models import *
2  class Usuario(models.Model):
3      user = models.OneToOneField(User, on_delete=models.CASCADE)
4      perfil = models.IntegerField(choices=PERFIL, default=2)
5      aniversario = models.DateField(default=None, null=True, blank=True)
6      criado_em = models.DateTimeField(auto_now_add=True)
7      alterado_em = models.DateTimeField(auto_now=True)
8      token = models.CharField(max_length=255, null=True, blank=True)
9      def __str__(self):
10         return '{}'.format(self.user.username)
11     @receiver(post_save, sender=User)
12     def create_user_usuario(sender, instance, created, **kwargs):
13         try:
14             if created:
15                 Usuario.objects.create(user=instance)
16         except:
17             pass
18     @receiver(post_save, sender=User)
19     def save_user_usuario(sender, instance, **kwargs):
20         try:
21             instance.usuario.save()
22         except:
23             pass
```

Django

Capítulo 8 - Trabalhando com Templates II

Explicando metodos:

create_user_usuario: método para criar o perfil do usuário. Para que isso ocorra automaticamente, precisamos passar a notação `@receiber(post_save, sender=User)` , onde `post_save` informa que, em um determinado método `post` , `create_user_usuario` deve ser acionado. Já `sender=User` informa qual classe `Model` está sendo chamada no `post`, sendo assim, quando houver um `post` na classe `User` , deverá ser chamado o método `create_user_usuario`. Não podemos esquecer que requisições `post` são para criações, então um `post` na classe `User` aciona o método `save` dela. No resumo, toda vez que ocorrer a criação de um novo `User`, um `Usuario` será criado para ele.

save_user_usuario: salva qualquer alteração no perfil automaticamente quando entramos no painel de perfil e alteramos qualquer dado.

Django

Capítulo 8 - Trabalhando com Templates II

Para incluir a classe usuário e a definição das opções de perfil definidas na propriedade choice do campo perfil, altere o arquivo loja\models__init__.py:

```
from django.db import models
from django.contrib.auth.models import User
from django.db.models.signals import post_save
from django.dispatch import receiver
from .Fabricante import Fabricante
from .Categoria import Categoria
from .Produto import Produto
# inclua o modelo de usuário
PERFIL = (
    (1, 'Admin'),
    (2, 'Usuario')
)
from .Usuario import Usuario
```



```
loja > models > __init__.py > ...
1  from django.db import models
2  from django.contrib.auth.models import User
3  from django.db.models.signals import post_save
4  from django.dispatch import receiver
5  from .Fabricante import Fabricante
6  from .Categoria import Categoria
7  from .Produto import Produto
8  # inclua o modelo de usuário
9  from .Usuario import Usuario
10 PERFIL = (
11     (1, 'Admin'),
12     (2, 'Usuario')
13 )
```

Django

Capítulo 8 - Trabalhando com Templates II

Execute o comando makemigrations para criar a migração do banco de dados que inclui a tabela da classe Usuario no banco de dados

```
python manage.py makemigrations loja
```

A quarta migração será criada na pasta de migrações

Execute o comando migrate para aplicar a migração ao banco de dados

```
python manage.py migrate loja
```

```
(venv) C:\Repos\django\thiago>python manage.py makemigrations loja
Migrations for 'loja':
  loja\migrations\0005_usuario.py
    - Create model Usuario
```

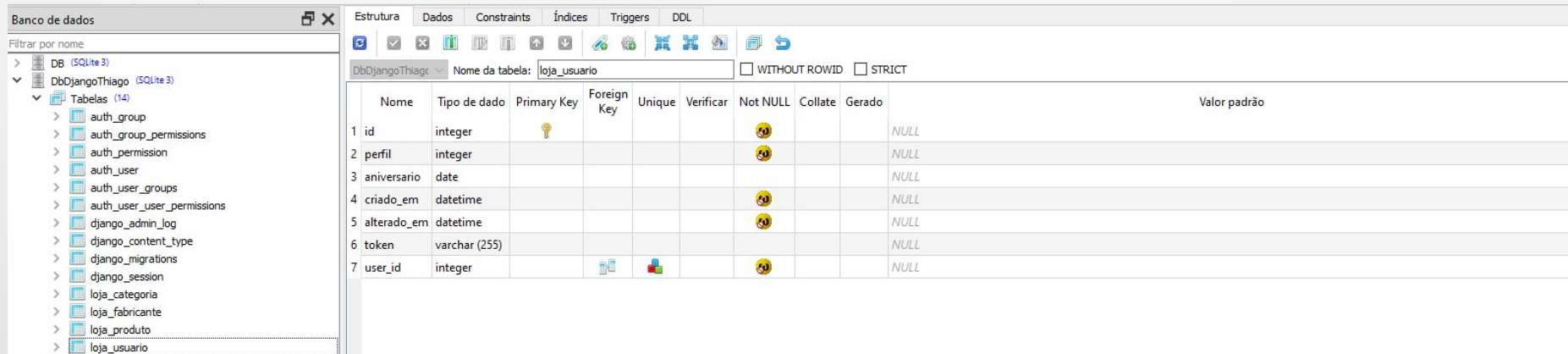
```
(venv) C:\Repos\django\thiago>
```

```
(venv) C:\Repos\django\thiago>python manage.py migrate loja
Operations to perform:
  Apply all migrations: loja
Running migrations:
  Applying loja.0005_usuario... OK
```

Django

Capítulo 8 - Trabalhando com Templates II

Abra o banco de dados para observar que a nova tabela consta na estrutura



The screenshot shows a database management interface with a sidebar on the left and a main table view on the right. The sidebar, titled 'Banco de dados', shows a tree structure of databases and tables. The main area, titled 'Estrutura', shows the structure of the 'loja_usuario' table. The table has 7 columns: Nome, Tipo de dado, Primary Key, Foreign Key, Unique, Verificar, Not NULL, Collate, Gerado, and Valor padrão. The rows are numbered 1 to 7.

	Nome	Tipo de dado	Primary Key	Foreign Key	Unique	Verificar	Not NULL	Collate	Gerado	Valor padrão
1	id	integer								NULL
2	perfil	integer								NULL
3	aniversario	date								NULL
4	criado_em	datetime								NULL
5	alterado_em	datetime								NULL
6	token	varchar (255)								NULL
7	user_id	integer								NULL

Django

Capítulo 8 - Trabalhando com Templates II

Abra o arquivo loja/**admin.py** e inclua o comando para registrar a nova tabela na interface do admin

Conteúdo do loja/**admin.py**:

```
from django.contrib import admin
from .models import *
class FabricanteAdmin(admin.ModelAdmin):
    #Conteúdo da classe
    admin.site.register(Fabricante, FabricanteAdmin)
    admin.site.register(Categoria)
class ProdutoAdmin(admin.ModelAdmin):
    #Conteúdo da classe
    admin.site.register(Produto, ProdutoAdmin)
# incluir a tabela de usuário no final
admin.site.register(Usuario)
```

Django

Capítulo 8 - Trabalhando com Templates II

Rode a aplicação, e acesse a página do admin;

Comando para rodar:

```
python manage.py runserver 127.0.0.1:8080
```

Link para acessar

<http://127.0.0.1:8080/admin>

```
(venv) C:\Repos\django\thiago>python manage.py runserver 127.0.0.1:8080
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 17, 2024 - 18:11:02
Django version 4.2.7, using settings 'lojaAdmin.settings.development'
Starting development server at http://127.0.0.1:8080/
Quit the server with CTRL-BREAK.
```

127.0.0.1:8080/admin/login/?next=/admin/

Administração do Django

Usuário:

admin

Senha:

.....

Acessar

Django

Capítulo 8 - Trabalhando com Templates II

A nova tabela de Usuario para os dados adicionais de usuário é acessível.

Administração do Django

Administração do Site

- AUTENTICAÇÃO E AUTORIZAÇÃO
 - Grupos
 - Usuários
- LOJA
 - Categorias
 - Fabricantes
 - Produtos
 - Usuarios

127.0.0.1:8080/admin/loja/usuario/

Administração do Django

BEM-VINDO(A), **ADMIN** VER O SITE / ALTERAR SENHA / ENCERRAR SESSÃO

Início > Loja > Usuarios

Comece a digitar para filtrar...

AUTENTICAÇÃO E AUTORIZAÇÃO

- Grupos + Adicionar
- Usuários + Adicionar

LOJA

- Categorias + Adicionar
- Fabricantes + Adicionar
- Produtos + Adicionar
- Usuarios + Adicionar

Selecione usuario para modificar

0 usuarios

ADICIONAR USUARIO +

Django

Capítulo 8 - Trabalhando com Templates II

Nela iremos poder controlar o perfil dos usuários e incluir qualquer informação adicional necessária a gestão de usuários específica da aplicação desenvolvida.

Como o modelo da tabela usuário foi ligado ao modelo User, ao criar um user um registro de usuário é inserido automaticamente.

Na classe modelo de usuário definimos o perfil padrão do usuário com o valor 2, ou seja, perfil de usuário(Usuario).

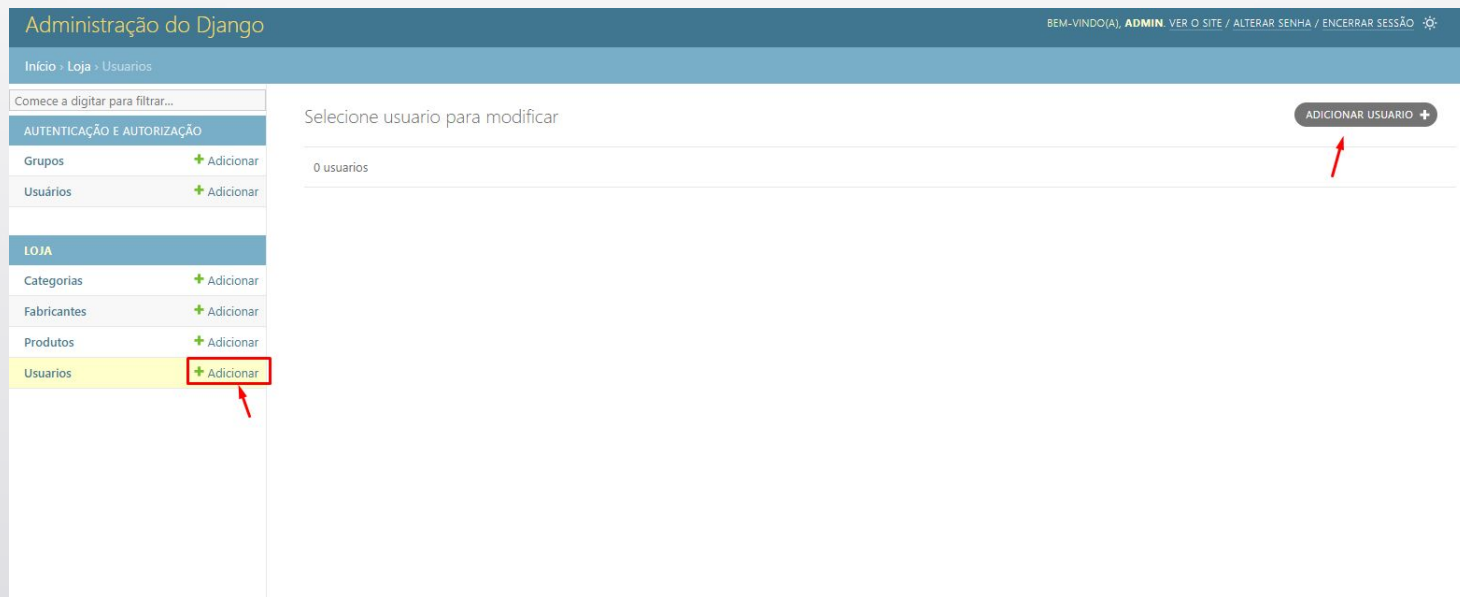
O valor 1 indica no exemplo que o usuário é administrador(Admin).

Por enquanto os usuários já criados não têm um registro na tabela de Usuário, crie mais 2 e adicione o admin na nossa tabela.

Django

Capítulo 8 - Trabalhando com Templates II

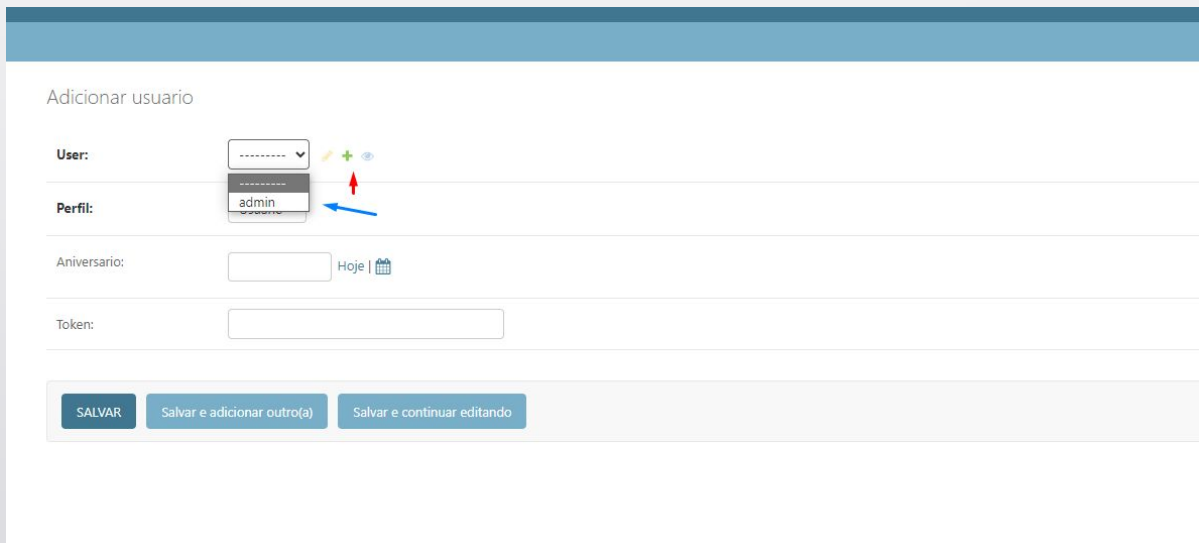
Para começar vá na nossa tabela e aperte no botão ADICIONAR USUARIO.



Django

Capítulo 8 - Trabalhando com Templates II

Na imagem abaixo, a seta azul aponta para seleção de usuários existentes, que podem ser vistos na tabela Usuários na seção AUTENTICAÇÃO E AUTORIZAÇÃO. A seta vermelha já permite inserir um Usuário direto a partir dessa interface, adicione dois usuários.



Adicionar usuário

User:

Perfil:

Aniversário:

Token:

Django

Capítulo 8 - Trabalhando com Templates II

Criando 2 usuários, usuario1 e usuario2

Adicionar usuário | Site de administração do Django - Trabalho — Microsoft Edge

127.0.0.1:8080/admin/auth/user/add/?_to_field=id&_popup=1

Adicionar usuário

Digite um nome de usuário e senha.

Usuário:

Obrigatório. 150 caracteres ou menos. Letras, números e @/./+/-/_ apenas.

Senha:

Sua senha não pode ser muito parecida com o resto das suas informações pessoais.
Sua senha precisa conter pelo menos 8 caracteres.
Sua senha não pode ser uma senha comumente utilizada.
Sua senha não pode ser inteiramente numérica.

Confirmação de senha:

Informe a mesma senha informada anteriormente, para verificação.

SALVAR

Adicionar usuário | Site de administração do Django - Trabalho — Microsoft Edge

127.0.0.1:8080/admin/auth/user/add/?_to_field=id&_popup=1

Adicionar usuário

Digite um nome de usuário e senha.

Usuário:

Obrigatório. 150 caracteres ou menos. Letras, números e @/./+/-/_ apenas.

Senha:

Sua senha não pode ser muito parecida com o resto das suas informações pessoais.
Sua senha precisa conter pelo menos 8 caracteres.
Sua senha não pode ser uma senha comumente utilizada.
Sua senha não pode ser inteiramente numérica.

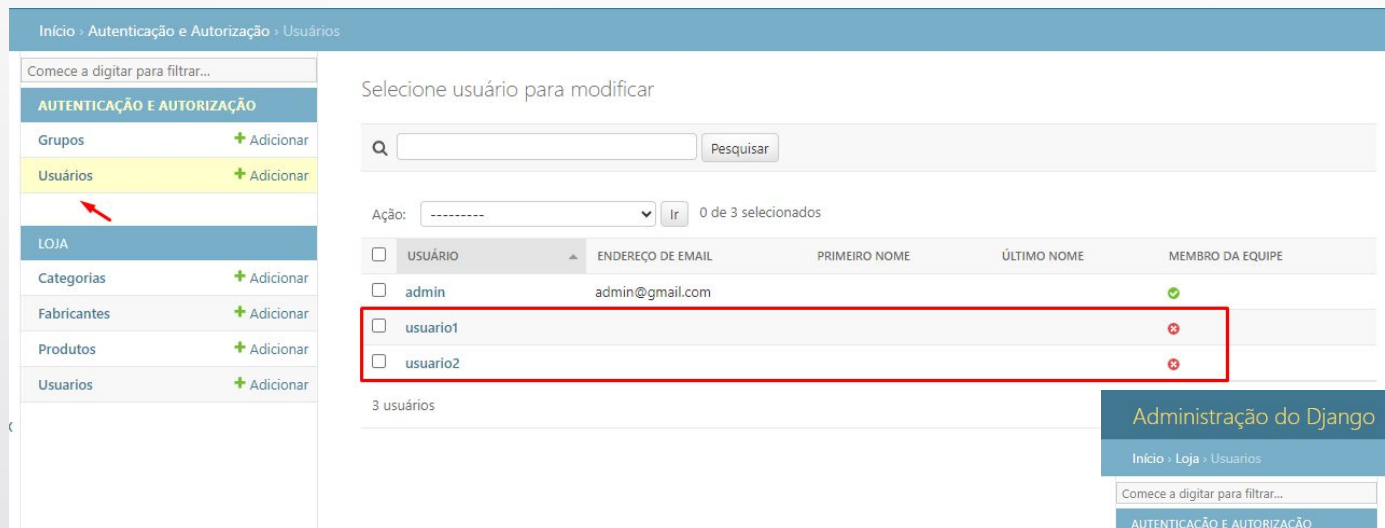
Confirmação de senha:

Informe a mesma senha informada anteriormente, para verificação.

SALVAR

Django

Capítulo 8 - Trabalhando com Templates II



Inicio > Autenticação e Autorização > Usuários

Comece a digitar para filtrar...

AUTENTICAÇÃO E AUTORIZAÇÃO

- Grupos + Adicionar
- Usuários + Adicionar**

LOJA

- Categorias + Adicionar
- Fabricantes + Adicionar
- Produtos + Adicionar
- Usuarios + Adicionar

Selecione usuário para modificar

Q [Pesquisar]

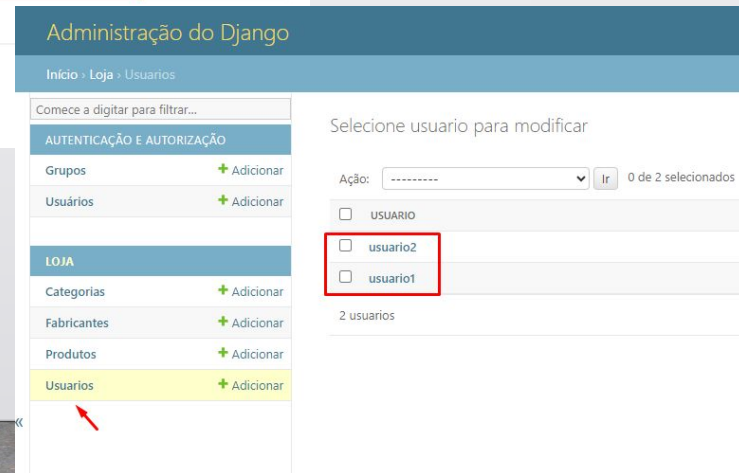
Ação: [-----] Ir 0 de 3 selecionados

☐	USUÁRIO	ENDEREÇO DE EMAIL	PRIMEIRO NOME	ÚLTIMO NOME	MEMBRO DA EQUIPE
☐	admin	admin@gmail.com			✓
☐	usuario1				✗
☐	usuario2				✗

3 usuários

A imagem do lado esquerdo mostra a tabela padrão do Django para usuários

A imagem do lado direito mostra a nossa tabela que serve como incremento para a tabela de usuário padrão do Django



Administração do Django

Inicio > Loja > Usuarios

Comece a digitar para filtrar...

AUTENTICAÇÃO E AUTORIZAÇÃO

- Grupos + Adicionar
- Usuários + Adicionar

LOJA

- Categorias + Adicionar
- Fabricantes + Adicionar
- Produtos + Adicionar
- Usuarios + Adicionar**

Selecione usuario para modificar

Ação: [-----] Ir 0 de 2 selecionados

☐	USUARIO
☐	usuario2
☐	usuario1

2 usuarios

Django

Capítulo 8 - Trabalhando com Templates II

A seta vermelha está apontando para a tabela que estou manipulando. A seta verde está apontando para minha ação no momento (edição do usuario2). A seta azul ta mostrando o valor inicial do usuário criado, que é não admin, ou seja, Usuario.

Comece a digitar para filtrar...

AUTENTICAÇÃO E AUTORIZAÇÃO

- Grupos + Adicionar
- Usuários + Adicionar

LOJA

- Categorias + Adicionar
- Fabricantes + Adicionar
- Produtos + Adicionar
- Usuarios + Adicionar**

Modificar usuario

usuario2

User: usuario2 ✎ + 👁

Perfil: Usuario ←

Aniversario: Hoje 📅

Token:

SALVAR Salvar e adicionar outro(a) Salvar e continuar editando

Django

Capítulo 8 - Trabalhando com Templates II

Adicionando um usuário na nossa tabela para o admin.

The screenshot displays the Django Admin interface. On the left is a sidebar menu with a search bar at the top. Below it, the 'AUTENTICAÇÃO E AUTORIZAÇÃO' section is expanded, showing 'Grupos' and 'Usuários', both with '+ Adicionar' links. The 'LOJA' section is also visible with links for 'Categorias', 'Fabricantes', 'Produtos', and 'Usuarios' (highlighted in yellow). The main content area is titled 'Adicionar usuario'. It contains four form fields: 'User:' with a dropdown set to 'admin' and icons for help, add, and view; 'Perfil:' with a dropdown set to 'Admin'; 'Aniversario:' with a date input set to '31/12/2005' and a 'Hoje' button with a calendar icon; and 'Token:' with an empty text input. At the bottom are three buttons: 'SALVAR', 'Salvar e adicionar outro(a)', and 'Salvar e continuar editando'.

Comece a digitar para filtrar...	
AUTENTICAÇÃO E AUTORIZAÇÃO	
Grupos	+ Adicionar
Usuários	+ Adicionar
LOJA	
Categorias	+ Adicionar
Fabricantes	+ Adicionar
Produtos	+ Adicionar
Usuarios	+ Adicionar

Adicionar usuario

User:

Perfil:

Aniversario: Hoje

Token:

Django

Capítulo 8 - Trabalhando com Templates II

Agora, criaremos o template que exibirá o perfil do usuário no sistema.

Crie uma pasta chamada `usuario` dentro da pasta `template` e dentro dela crie um arquivo `usuario.html`. Essa interface irá mostrar a lista de usuários.

Parte 1 do conteúdo do `loja/templates/usuario/usuario.html` :

```
{% extends "base.html" %}
{% load static %}
{% block title %}Início{% endblock %}
{% block styles %}
    <link rel="stylesheet" href="{% static 'css/home.css' %}">
{% endblock %}
{% block content %}
<div id="content">
    <div class="container">
        <div class="alert alert-info">
            Usuários encontrados [{{ usuarios | length }}]
        </div>
```

Django

Capítulo 8 - Trabalhando com Templates II

Parte 2 do conteúdo do loja/templates/usuário/**usuario.html** :

```
<div class="row">
    <h5 class="col-md-2">Usuário</h5>
    <h5 class="col-md-4">e-mail</h5>
    <h5 class="col-md-3">Aniversário</h5>
</div>
{% for usuario in usuarios %}
    <div class="row">
        <h5 class="col-md-2">{{usuario.user.username}}</h5>
        <h5 class="col-md-4">{{usuario.user.email}}</h5>
        <h5 class="col-md-3">{{usuario.aniversario}}</h5>
    </div>
{% endfor %}
</div>
{% endblock %}{% block scripts %}<script src="{% static 'js/home.js'%}"></script>{%
endblock %}
```

Django

Capítulo 8 - Trabalhando com Templates II

Imagem do código do loja/templates/usuário/**usuario.html** :

```
loja > templates > usuario > <> usuario.html > ...
1  {% extends "base.html" %}
2  {% load static %}
3  {% block title %}Inicio{% endblock %}
4  {% block styles %}
5  <link rel="stylesheet" href="{% static 'css/home.css' %}">
6  {% endblock %}
7  {% block content %}
8  <div id="content">
9      <div class="container">
10         <div class="alert alert-info">
11             Usuários encontrados [{{ usuarios | length }}]
12         </div>
13         <div class="row">
14             <h5 class="col-md-2">Usuário</h5>
15             <h5 class="col-md-4">e-mail</h5>
16             <h5 class="col-md-3">Aniversário</h5>
17         </div>
18         {% for usuario in usuarios %}
19         <div class="row">
20             <h5 class="col-md-2">{{usuario.user.username}}</h5>
21             <h5 class="col-md-4">{{usuario.user.email}}</h5>
22             <h5 class="col-md-3">{{usuario.aniversario}}</h5>
23         </div>
24         {% endfor %}
25     </div>
26 </div>
27 {% endblock %}
28 {% block scripts %}
29 <script src="{% static 'js/home.js' %}"></script>
30 {% endblock %}
```

Django

Capítulo 8 - Trabalhando com Templates II

Agora para criar a camada de visão, adicione dentro da pasta views o arquivo **UsuarioView.py**. Nele inclua o comando para carregar os dados dos usuários.

Conteúdo do loja/views/**UsuarioView.py**

```
from django.shortcuts import render, redirect
from loja.models import Usuario
def list_usuario_view(request, id=None):
    # carrega somente usuarios, não inclui os admin
    usuarios = Usuario.objects.filter(perfil=2)

    context = {
        'usuarios': usuarios
    }
    return render(request, template_name='usuario/usuario.html', context=context,
status=200)
```

Django

Capítulo 8 - Trabalhando com Templates II

Agora vamos criar o arquivo de urls para definir a rota que irá chamar a interface de usuários. Inclua na pasta loja/urls o arquivo **UsuarioUrls.py**.

Parte 1 do conteúdo do loja/urls/**UsuarioUrls.py**:

```
from django.urls import path
from loja.views.UsuarioView import list_usuario_view
urlpatterns = [
    path("", list_usuario_view, name='usuario'),
]
```



```
loja > urls > UsuarioUrls.py > ...
1  from django.urls import path
2  from loja.views.UsuarioView import list_usuario_view
3  urlpatterns = [
4      path("", list_usuario_view, name='usuario'),
5  ]
```

Django

Capítulo 8 - Trabalhando com Templates II

E no arquivo **__init__.py** da pasta `Urls` importe o arquivo `UsuarioUrls.py`, assim:

```
from .HomeUrls import *  
from .ProdutoUrls import *  
from .UsuarioUrls import *
```

loja > urls >  __init__.py

```
1  ✓ from .HomeUrls import *  
2    from .ProdutoUrls import *  
3    from .UsuarioUrls import *
```

Django

Capítulo 8 - Trabalhando com Templates II

Inclua no arquivo LojaAdmin\Urls.py as urls de usuário

```
from django.contrib import admin
from django.urls import path
from django.conf.urls.static import static
from django.conf import settings
from django.conf.urls import include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('loja.urls.HomeUrls')),
    path('produto/', include('loja.urls.ProdutoUrls')),
    # Adicione a linha a seguir
    path('usuario/', include('loja.urls.UsuarioUrls')),
] + static(settings.STATIC_URL, document_root=settings.STATIC_ROOT) +
static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

lojaAdmin >  urls.py > ...

```
1 from django.contrib import admin
2 from django.urls import path
3 from django.conf.urls.static import static
4 from django.conf import settings
5 from django.conf.urls import include
6 urlpatterns = [
7     path('admin/', admin.site.urls),
8     path('', include('loja.urls.HomeUrls')),
9     path('produto/', include('loja.urls.ProdutoUrls')),
10    # Adicione a linha a seguir
11    path('usuario/', include('loja.urls.UsuarioUrls')),
12 ] + static(settings.STATIC_URL, document_root=settings.STA
```

Django

Capítulo 8 - Trabalhando com Templates II

Inclua no menu da aplicação, no arquivo loja/templates/**base.html**, o botão abaixo para acessar o cadastro de usuários antes do botão de entrar e depois do de produto, no nav do header.

```
{% load static %}
<!DOCTYPE html>
[...]<header>
[...]  
<li class="nav-item"><a class="nav-link" href="/produto"><i class="fa fa-book"></i>  
Produto</a></li>  
<!--inclua o botão de acesso ao cadastro de usuário-->  
<li class="nav-item"><a class="nav-link" href="/usuario"><i class="fa fa-book"></i>  
Usuário</a></li>  
<li class="nav-item"><a class="nav-link" href="#"><i class="fa fa-sign-in"></i>  
Entrar</a></li>  
[...]  
<header>[...]
```

Django

Capítulo 8 - Trabalhando com Templates II

Rode a aplicação e acesse a listagem

